

Rapport de Stage de Fin d'Études

Pour l'obtention du Diplôme d'Ingénieur d'État et du Master

Conception et mise en œuvre d'un pipeline automatisé de contrôle de la qualité des données : Cas des données CSP et KYC

Filière :

Réalisé par :

SAAD BENDAHOUE

MIAGE (Méthodes Informatiques
Appliquées à la Gestion des
Entreprises)
MIAGE I2A (Informatique et
Intelligence Artificielle)

Encadrant de l'École :

Mr. Marouan Skiti

Encadrants de CIH BANK :

Mr. Mohamed Agouzzal

Mr. Marouan Naoui

Année Universitaire : 2025 – 2026

Dédicaces

Je dédie ce modeste travail :

À mes chers parents,

Aucun mot, aucune dédicace ne saurait exprimer mon amour, mon respect et ma profonde gratitude pour vos sacrifices, votre soutien inconditionnel et vos prières qui m'ont toujours accompagné tout au long de mon parcours universitaire. Que Dieu vous préserve et vous procure santé et longue vie.

À mes frères et sœurs,

Pour votre présence constante, vos encouragements et votre affection fraternelle.

À toute ma famille, mes proches et mes amis,

Qui ont toujours cru en moi et m'ont soutenu durant les moments les plus intenses de mes études.

Merci d'être toujours là pour moi.

Remerciements

Avant tout, je tiens à exprimer ma profonde gratitude à Dieu le Tout-Puissant pour m'avoir donné la force, la patience et la persévérance nécessaires pour mener à bien ce travail.

Je tiens à exprimer mes sincères remerciements à mon école d'accueil, l'Université de Nice Sophia Antipolis (UNICA) et l'École Marocaine des Sciences de l'Ingénieur (EMSI) pour m'avoir assuré une excellente formation académique et des bases solides pour entamer ma vie professionnelle.

J'adresse mes remerciements les plus chaleureux à mon encadrant pédagogique, **Mr. Marouan Skiti**, pour ses conseils méthodologiques avisés, sa disponibilité continue et la bienveillance dont il a fait preuve tout au long du suivi de ce projet de fin d'études.

Mes remerciements les plus vifs s'adressent à l'institution de mon stage, **CIH BANK**, pour m'avoir ouvert ses portes et permis de réaliser ce travail dans un environnement professionnel stimulant et formateur.

Je exprime toute ma reconnaissance à mes encadrants de stage chez CIH Bank, **Mr. Mohamed Agouzzal** (Manager du Data Office) et **Mr. Marouan Naoui** (Data Engineer / Steward), pour leur accueil chaleureux, leur confiance, leur précieux encadrement et leur soutien technique rigoureux durant toute la durée de mon projet. Leurs directives éclairées m'ont grandement aidé à surmonter les difficultés techniques et à concevoir ce pipeline.

Enfin, je remercie l'ensemble des collaborateurs de CIH Bank et les membres de l'équipe Data Office pour leur esprit d'équipe, leur soutien et les échanges enrichissants que nous avons partagés. Que tous ceux qui ont contribué de près ou de loin à la réussite de ce projet trouvent ici l'expression de ma profonde gratitude.

Résumé

Dans un contexte de transformation digitale du secteur bancaire, la donnée constitue un levier stratégique essentiel pour la prise de décision, la gestion des risques et l'amélioration de la performance globale. Cependant, la qualité des données représente un enjeu majeur, car des données incomplètes, incohérentes ou erronées peuvent impacter significativement les processus métiers et les analyses.

Ce projet s'inscrit dans le cadre des initiatives de Data Governance au sein de CIH Bank. Il vise à concevoir et mettre en œuvre un pipeline automatisé de contrôle de la qualité des données, permettant d'assurer la fiabilité, la traçabilité et la cohérence des informations exploitées.

La solution proposée repose sur l'extraction et le traitement de données provenant de différentes sources, notamment le Data Lake et des fichiers structurés, en intégrant des règles de contrôle couvrant plusieurs dimensions de la qualité des données telles que la complétude, la validité, l'unicité et l'intégrité.

L'automatisation des contrôles permet d'instaurer une surveillance continue de la qualité des données, avec la production d'indicateurs (KPI) facilitant le pilotage et la prise de décision. Ce projet contribue ainsi à renforcer la gouvernance des données et à soutenir les initiatives analytiques et décisionnelles de la banque.

Abstract

In the context of digital transformation within the banking sector, data has become a key strategic asset for decision-making, risk management, and overall performance improvement. However, data quality remains a major challenge, as incomplete, inconsistent, or erroneous data can significantly impact business processes and analytical outcomes.

This project is part of the Data Governance initiatives at CIH Bank. Its objective is to design and implement an automated data quality control pipeline to ensure the reliability, traceability, and consistency of the data being used.

The proposed solution is based on processing data from multiple sources, including the Data Lake and structured files, while applying a set of rules covering key data quality dimensions such as completeness, validity, uniqueness, and integrity.

By automating these controls, the solution enables continuous monitoring of data quality and the generation of key performance indicators (KPIs), supporting efficient data governance and informed decision-making. This project therefore contributes to strengthening the organization's data management framework and enhancing its analytical capabilities.

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des abréviations	x
Introduction Générale	1
1 Présentation du cadre de projet	2
1.1 Présentation de l'organisme d'accueil	2
1.1.1 Missions de CIH BANK	2
1.1.2 Secteurs d'activité	3
1.1.3 Filiales	3
1.1.4 Actionnariat	3
1.1.5 Chiffres clés	3
1.1.6 Organigramme	4
1.1.7 Data Office	5
1.2 Étude de l'existant	6
1.2.1 Description de l'existant	6
1.2.2 Le domaine Tiers : périmètre métier du projet	6
1.2.3 Le pipeline de données existant : de NOVA à Tableau	7
1.2.4 Critique de l'existant	9
1.2.5 Analyse de la problématique technique	10
1.2.6 Solution proposée	10
1.2.7 Solution proposée : automatisation de la Data Quality	11
1.3 Choix du modèle de développement	12
1.4 Planning prévisionnel	13
2 Analyse des Besoins et Spécification	14
2.1 Besoins fonctionnels	14
2.1.1 Automatisation et planification des contrôles	14
2.1.2 Configuration déclarative des règles de qualité	14
2.1.3 Historisation et traçabilité des indicateurs	15
2.1.4 Restitution visuelle et aide à la décision	15
2.2 Besoins non fonctionnels	15
2.2.1 Performance et passage à l'échelle (Scalabilité)	15
2.2.2 Robustesse et découplage (Tolérance aux pannes)	15

2.2.3	Maintenabilité et extensibilité	15
2.2.4	Sécurité et gouvernance	16
2.3	Modélisation des cas d'utilisation	16
2.3.1	Acteurs du système	16
2.3.2	Diagramme global des cas d'utilisation	16
2.3.3	Description textuelle des cas d'utilisation principaux	17
3	Conception de la Solution	19
3.1	Architecture globale de la solution	19
3.1.1	Vue d'ensemble	19
3.1.2	Outils et technologies	20
3.2	Catalogue des règles de Data Quality	22
3.2.1	Principes de conception des règles	22
3.2.2	Règles DQ sur les colonnes KYC	22
3.2.3	Règles DQ sur les colonnes CSP	23
3.2.4	Règles DQ Pipeline (Fraîcheur)	24
3.3	Modèle de données DQ (Persistance)	24
3.3.1	Base Hive <code>dq_results</code>	24
3.3.2	Table <code>dq_tiers_scores</code> — Historique des scores	24
3.3.3	Table <code>dq_tiers_anomalies</code> — Détail des anomalies	25
3.3.4	Partitionnement et format de stockage	25
3.4	Conception du moteur DQ — Soda Core	26
3.4.1	Architecture du script <code>dq_tiers_checks.py</code>	26
3.4.2	Fichiers de configuration YAML	26
3.5	Conception de l'orchestration Airflow	26
3.5.1	Intégration dans le DAG <code>dailyow22</code>	26
3.6	Conception des vues Dremio DQ	26
3.7	Conception du dashboard Tableau	27
4	Réalisation de la Solution	28
4.1	Environnement technique de développement	28
4.1.1	Environnement Hadoop et cluster	28
4.1.2	Installation de Soda Core	29
4.2	Implémentation du moteur DQ — Soda Core	29
4.2.1	Structure des fichiers du projet DQ	29
4.2.2	Fichier de configuration YAML — Règles KYC (<code>checks_kyc.yml</code>)	29
4.2.3	Fichier de configuration YAML — Règles CSP (<code>checks_csp.yml</code>)	30
4.2.4	Script Python principal (<code>dq_tiers_checks.py</code>)	31
4.3	Création des tables Hive de persistance	33
4.4	Intégration dans le DAG Airflow <code>dailyow22</code>	34
4.4.1	Ajout de la tâche DQ	34
4.4.2	Planification et monitoring	34
4.5	Création des vues Dremio	34
4.5.1	Vue <code>v.dq_scores_history</code>	35
4.5.2	Vue <code>v.dq_kpis_summary</code>	35
4.5.3	Vue <code>v.dq_anomalies_detail</code>	35
4.6	Réalisation du dashboard Tableau	36
4.6.1	Connexion Tableau à Dremio	36
4.6.2	Onglet 1 — Vue Exécutive	36
4.6.3	Onglet 2 — Détail KYC	37

4.6.4	Onglet 3 — Détail CSP	37
4.6.5	Onglet 4 — Anomalies et Remédiation	37

Table des figures

1.1	Chiffres clés de CIH BANK (31/12/2024)	4
1.2	Organigramme de CIH BANK	5
1.3	Architecture du pipeline de données Référentiel Tiers PP Clients	8
1.4	Diagramme de Gantt du planning prévisionnel du projet	13
2.1	Diagramme global des cas d'utilisation de la solution DQ	17
3.1	Architecture globale de la solution de Data Quality	20
3.2	Positionnement de la tâche DQ dans le DAG dailyow22	26
4.1	Dashboard Tableau — Vue Exécutive : scores DQ globaux KYC et CSP . .	36
4.2	Dashboard Tableau — Détail KYC : score par colonne et dimension DQ .	37
4.3	Dashboard Tableau — Détail CSP : taux de nullité et valeurs parasites . .	37

Liste des tableaux

1.1	Informations générales sur le groupe CIH BANK [10]	2
1.2	Répartition du capital de CIH BANK [4]	3
1.3	Colonnes KYC du Référentiel Tiers PP	7
1.4	Colonnes CSP du Référentiel Tiers PP	7
3.1	Outils et technologies de la solution DQ Tiers	21
3.2	Règles DQ — Colonnes KYC	22
3.3	Règles DQ — Colonnes CSP	23
3.4	Règles DQ — Fraîcheur et Pipeline	24
3.5	Modèle de la table <code>dq_results.dq_tiers_scores</code>	25
3.6	Modèle de la table <code>dq_results.dq_tiers_anomalies</code>	25
3.7	Vues Dremio du schéma <code>dq-tiers</code>	27
4.1	Versions des composants de l'environnement technique	28

Liste des abréviations

Abréviation	Signification
CIH	Crédit Immobilier et Hôtelier
CDO	Chief Data Officer
DWH	Data Warehouse
BI	Business Intelligence
ETL	Extract, Transform, Load
SQL	Structured Query Language
KPI	Key Performance Indicator
API	Application Programming Interface
CSV	Comma-Separated Values
ANRF	Autorité Nationale du Renseignement Financier
CNSS	Caisse Nationale de Sécurité Sociale
CSP	Catégories Socio-Professionnelles
KYC	Know Your Customer

Introduction Générale

Au cours des dernières années, le secteur bancaire a connu une transformation profonde sous l'impulsion de la digitalisation et de l'essor des technologies liées à la donnée. Dans ce contexte, les institutions financières s'appuient de plus en plus sur leurs systèmes d'information pour analyser les comportements clients, optimiser leurs processus et piloter leurs activités de manière efficace.

La donnée est ainsi devenue un actif stratégique. Toutefois, sa valeur dépend directement de sa qualité. Des données inexactes ou mal structurées peuvent engendrer des erreurs d'analyse, des décisions inadaptées et des risques importants, notamment dans des domaines sensibles tels que l'octroi de crédit ou la conformité réglementaire.

Face à ces enjeux, les organisations mettent en place des démarches de Data Governance visant à structurer, contrôler et valoriser leurs données. Parmi les axes majeurs de cette gouvernance figure la gestion de la qualité des données, qui repose sur la définition de règles de contrôle, leur automatisation et le suivi d'indicateurs de performance.

C'est dans ce cadre que s'inscrit le présent projet, réalisé au sein de CIH Bank. Il a pour objectif de concevoir et de mettre en œuvre une solution automatisée permettant de contrôler, mesurer et améliorer la qualité des données.

Le présent rapport est structuré comme suit : le premier chapitre présente l'organisme d'accueil, le deuxième expose le contexte et la problématique, le troisième décrit la conception de la solution, le quatrième détaille sa réalisation, et le dernier chapitre analyse les résultats obtenus.

Chapitre 1

Présentation du cadre de projet

Introduction

Dans ce chapitre, nous présenterons une description générale de l'organisme d'accueil, CIH BANK, d'un point de vue organisationnel. Nous aborderons ensuite l'étude de l'existant, le choix du modèle de développement ainsi que le planning prévisionnel du projet, afin de mieux contextualiser notre travail de fin d'études.

1.1 Présentation de l'organisme d'accueil

Le Crédit Immobilier et Hôtelier (CIH), plus communément appelé CIH BANK, est une banque marocaine dont le siège social se trouve à Casablanca. Historiquement connu pour son activité dans le secteur de l'immobilier, le CIH propose désormais des services bancaires variés à ses clients, particuliers et professionnels. [10]

CIH BANK a été fondée en 1920 sous le nom de Caisse de Prêts Immobiliers du Maroc (CPIM). En 1967, le groupe a étendu son activité au secteur hôtelier et a changé de nom pour devenir le Crédit Immobilier et Hôtelier. [10]

Création	1920
Fondateur	L'État marocain
Introduction en bourse	1967
Forme juridique	Société anonyme
Slogan	La banque de demain dès aujourd'hui
Siège social	187, Avenue Hassan II, Casablanca, Maroc
Président Directeur Général (PDG)	Lotfi SEKKAT
Société mère	Caisse de Dépôt et de Gestion
Effectif	2 181
Site web	www.cihbank.ma

TABLE 1.1 – Informations générales sur le groupe CIH BANK [10]

1.1.1 Missions de CIH BANK

Spécialisé dans le financement immobilier et hôtelier, le CIH est une banque à vocation particulière :

- Elle est un partenaire des pouvoirs publics en matière de financement du logement depuis 1920.

- Elle finance les promoteurs immobiliers au Maroc, par deux canaux différents :
 - En amont : Par le financement des promoteurs publics et privés (acquisition et viabilisation des terrains ainsi que la construction des programmes immobiliers).
 - En aval : Par l'octroi de crédits au logement pour les particuliers acquéreurs, sans oublier les services et produits proposés par la banque pour sa clientèle.

1.1.2 Secteurs d'activité

Le groupe CIH BANK et ses filiales offrent une vaste gamme de produits et de services à ses clients, particuliers et professionnels :

- Services bancaires ,
- Prêts aux particuliers (prêts immobiliers, crédits à la consommation, etc.) ,
- Financement d'entreprises (prêts d'investissement et d'exploitation, crédit-bail, etc.) ,
- Assurance ,
- Salle des marchés.

1.1.3 Filiales

Le groupe CIH BANK est constitué de plusieurs filiales :

- SOFAC : entreprise spécialisée dans les solutions de crédit ,
- SOFASSUR : entreprise spécialisée dans les services d'assurance ,
- Maroc Leasing : entreprise de leasing qui propose des solutions de financement pour les véhicules, le BTP, l'immobilier et la santé [6] ,
- CIH Courtage : courtier en assurance ,
- Umnia Bank : banque participative ,
- AJAR INVEST : société de gestion des Organismes de Placement Collectif Immobilier (OPCI), dont CIH BANK détient 40 % des parts [1] .

1.1.4 Actionnariat

La répartition du capital du groupe CIH BANK est présentée dans le tableau 1.2 :

TABLE 1.2 – Répartition du capital de CIH BANK [4]

Actionnaire	Part
Massira Capital Management	55,66 %
Flottant en bourse	16,37 %
Atlanta Sanad	11,61 %
Caisse de Dépôt et de Gestion (CDG)	6,69 %
Régime Collectif d'Allocation de Retraite (RCAR)	5,21 %
Personnel de la banque	4,36 %
Groupe Holmarcom	0,12 %

1.1.5 Chiffres clés

La figure 1.1 présente les chiffres clés de CIH BANK.

Les chiffres clés de CIH BANK reflètent la situation financière et opérationnelle de l'institution. Avec un capital social de 3,149 milliards de dirhams et un total bilan de 141 milliards de dirhams, la banque dispose d'une base financière significative. L'encours des

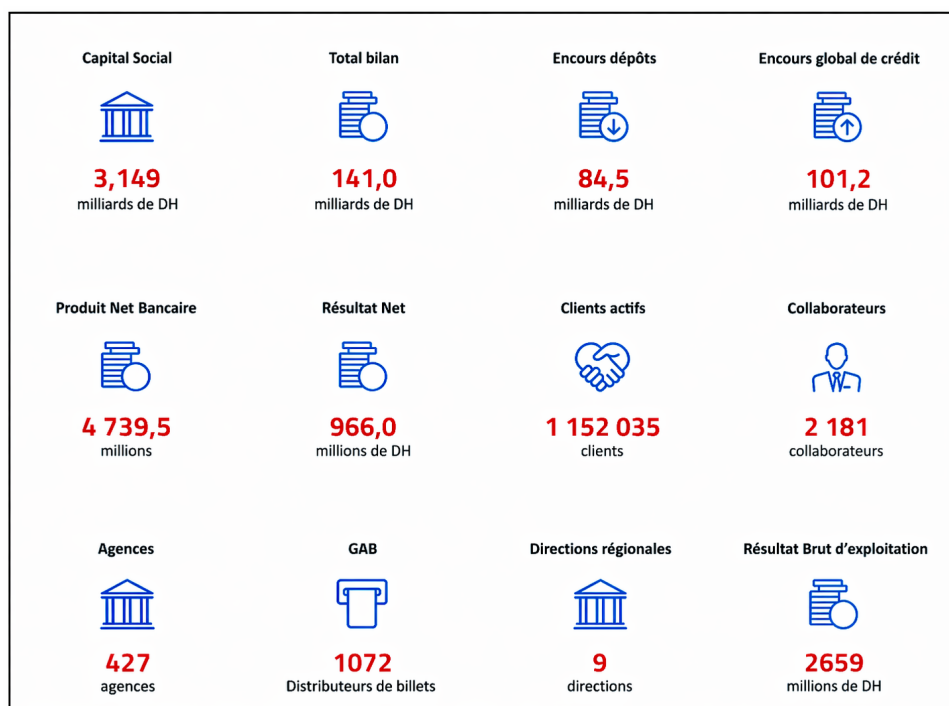


FIGURE 1.1 – Chiffres clés de CIH BANK (31/12/2024)

dépôts s'élève à 84,5 milliards de dirhams, tandis que l'encours global de crédit atteint 101,2 milliards de dirhams. Le produit net bancaire est de 4 739,5 millions de dirhams, avec un résultat net de 966 millions de dirhams et un résultat brut d'exploitation de 2 659 millions de dirhams. La banque compte plus de 1,15 million de clients actifs, 2 181 collaborateurs, 427 agences, 1 072 guichets automatiques et 9 directions régionales. Ces indicateurs fournissent une vue d'ensemble de l'activité, de la structure et des résultats financiers de l'institution.

1.1.6 Organigramme

Structure hiérarchique de CIH BANK

La structure hiérarchique de CIH BANK s'articule autour d'une direction générale pilotée par le Président Directeur Général, appuyée par des fonctions de support stratégique et de contrôle (Audit, Conformité, Risques, Capital Humain, etc.) ainsi que diverses fonctions rattachées. L'activité commerciale et opérationnelle de la banque est quant à elle répartie en plusieurs pôles métiers majeurs : la Banque des Particuliers et Professionnels, la Banque de l'Entreprise et de l'Immobilier, la Banque de l'Investissement, le Financement & Recouvrement, et les Services à la Clientèle & Réseaux.

Parmi ces différentes directions, notre attention se porte plus particulièrement sur l'entité technologique qui encadre notre organisme d'accueil :

Fonctions technologiques et data

- Services Technologiques et Opérations : D. BENNOUNA
- Sécurité des Systèmes d'Information ;
- Qualité ;
- Pôle Systèmes d'Information : T. CHARKAOUI ;
- Data Management & Intelligence Artificielle :
 - Data Office.

L'organigramme global de CIH BANK est illustré dans la figure 1.2 ci-dessous.

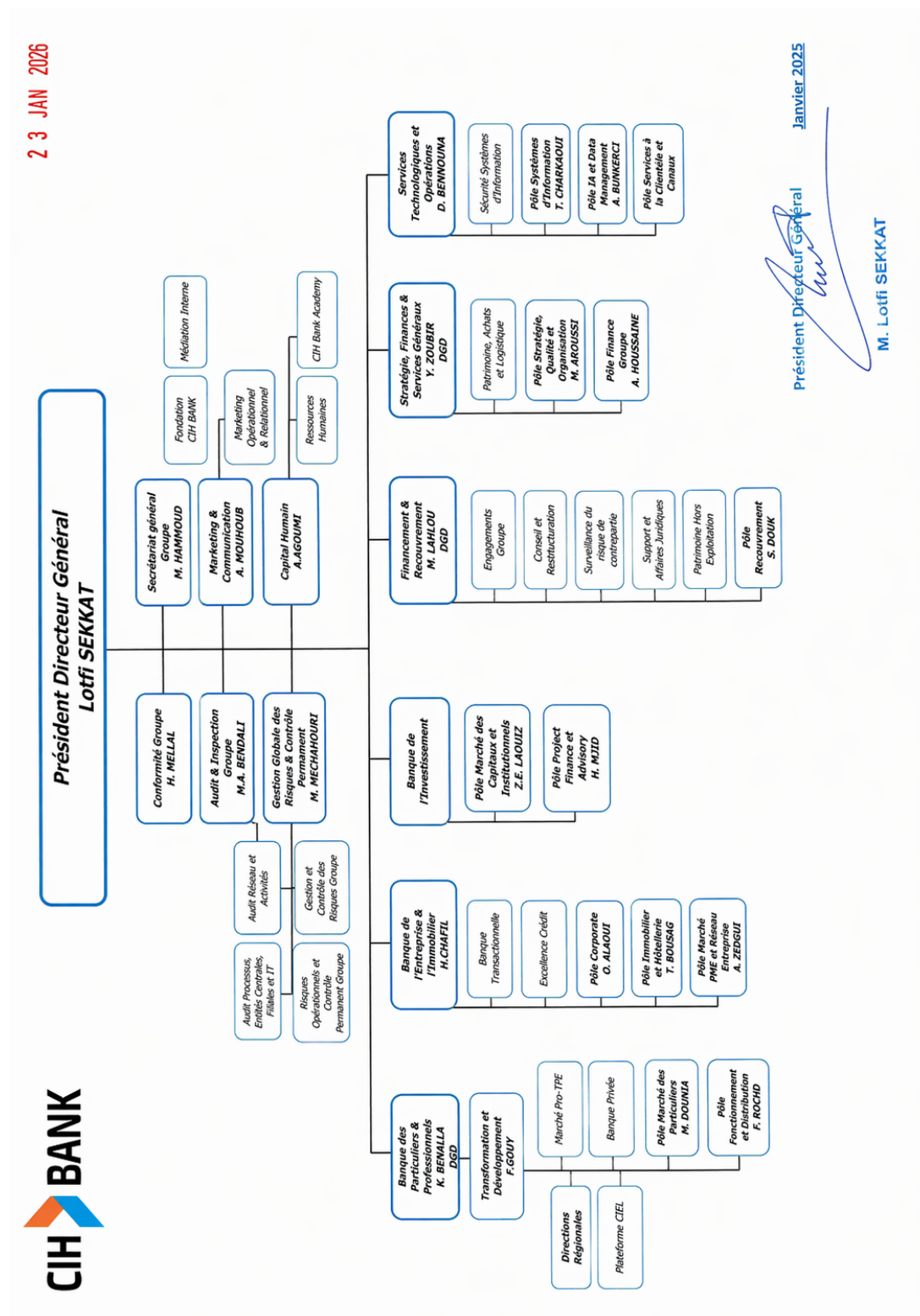


FIGURE 1.2 – Organigramme de CIH BANK

1.1.7 Data Office

Dans le cadre de notre stage, nous avons intégré le Data Office de CIH BANK, une entité dédiée à la gestion, au traitement et à la valorisation des données de la banque.

Le Data Office est aujourd'hui rattaché au pôle Services Technologiques et Opérations,

plus précisément au sein de la direction Data Management et Intelligence Artificielle. Cette organisation reflète l'importance stratégique accordée à la donnée dans le pilotage des activités de la banque.

Cette entité regroupe plusieurs profils complémentaires intervenant sur l'ensemble du cycle de vie de la donnée, notamment :

- Les Data Engineers, en charge de l'ingestion, du traitement et de la mise à disposition des données ,
- Les Data Analysts, responsables de l'analyse des données et de la production d'indicateurs décisionnels ,
- Les Data Scientists, spécialisés dans la modélisation avancée et l'analyse prédictive ,
- L'équipe Data Management, qui assure la gouvernance, la qualité et la structuration des données .

Les données sont collectées à partir des différents systèmes opérationnels de la banque, puis intégrées et centralisées au sein d'un Data Lake. Elles sont ensuite transformées et mises à disposition pour répondre aux besoins analytiques, réglementaires et métiers.

Dans ce cadre, le Data Office joue un rôle central dans la mise en place des pratiques de gestion de la qualité des données.

1.2 Étude de l'existant

1.2.1 Description de l'existant

Historiquement, la vérification de la qualité des données au sein de CIH BANK ne reposait sur aucun processus formel ni outil automatisé. L'absence d'un tel contrôle a d'ailleurs motivé la création récente de l'entité Data Management (DM). En pratique, le processus était exclusivement réactif et déclenché sur demande. Lorsqu'une anomalie ou une incohérence (notamment sur les données relatives au KYC ou aux Catégories Socioprofessionnelles - CSP) était constatée par les équipes métiers lors de l'exploitation de la donnée, une réclamation était émise. L'équipe Data Project prenait alors le relais en collaborant étroitement avec les Data Engineers. La résolution de ces incidents nécessitait souvent l'intervention d'autres départements du Système d'Information (SI), voire des métiers eux-mêmes, afin d'effectuer des tâches de réconciliation manuelles de manière curative.

1.2.2 Le domaine Tiers : périmètre métier du projet

Définition du Référentiel Tiers

Le **Référentiel Tiers** constitue le référentiel central de CIH Bank regroupant l'ensemble des informations relatives aux personnes physiques (PP) et morales (PM) en relation avec la banque. Ce référentiel est la source de vérité pour toutes les applications métiers : les crédits, la comptabilité, le reporting réglementaire et les outils de CRM.

Notre projet se concentre sur le périmètre des **Personnes Physiques (PP) Clients**, qui regroupe l'ensemble des clients particuliers et MRE (Marocains Résidant à l'Étranger) actifs de CIH Bank, soit plus de plusieurs millions de clients. Le filtre appliqué en production est : `customer_type = 1` et `code_marche IN (1, 8)`.

Les colonnes KYC et CSP : données critiques du Référentiel Tiers

Les colonnes ciblées par notre projet de Data Quality sont réparties en deux grands domaines :

Données KYC (*Know Your Customer*)

Les données KYC sont les informations d'identification réglementaires obligatoires pour chaque client. Elles sont issues de la table CCM_NPDESCRIPTION du système Oracle NOVA et comprennent notamment :

Colonne NOVA	Colonne CIHONE	Rôle métier
NOM	nom	Nom de famille du client
PRENOM	prenom	Prénom du client
DATE_NAISSANCE	date_de_naissance	Date de naissance
CIVILITE	civilite	Civilité : M., MME, MLLE
GENRE	genre	Genre : M ou F
CODE_NATIONALITE	code_pays_de_nationalite	Code pays de nationalité
CODE_PAYS_RESIDENCE	code_pays_de_residence	Pays de résidence
IDENTIFIANT_FATCA	identifiant_fatca	Numéro FATCA (clients US)
DATE_IMPRESSION_KYC	—	Date impression fiche KYC

TABLE 1.3 – Colonnes KYC du Référentiel Tiers PP

Données CSP (*Catégorie Socio-Professionnelle*)

Les données CSP décrivent la situation professionnelle du client. Elles proviennent de la table CCM_JOB et comprennent :

Colonne NOVA	Colonne CIHONE	Rôle métier
CODE_SECTEUR_PRO	code_secteur_professionnel	Code secteur (ex : 46=Agri-culture)
CODE_PROFESSION	code_profession	Code profession détaillée
CODE_CSP	code_categorie_socio_professionnelle	Catégorie socio-professionnelle
SITUATION_PRO	situation_professionnelle	SAL, FON, TRI, AUT, EMP, NDE
EMPLOYEUR	employeur	Nom de l'employeur
PPE	—	Personne Politiquement Ex-posée
NUM_CNSS	—	Numéro de sécurité sociale

TABLE 1.4 – Colonnes CSP du Référentiel Tiers PP

1.2.3 Le pipeline de données existant : de NOVA à Tableau

Vue d'ensemble de l'architecture

Les données du Référentiel Tiers suivent un parcours en plusieurs étapes, depuis le système opérant Oracle NOVA jusqu'aux outils de visualisation, en passant par un Data

Lake Hadoop. Le schéma ci-dessous décrit l'architecture globale de ce flux de bout en bout.

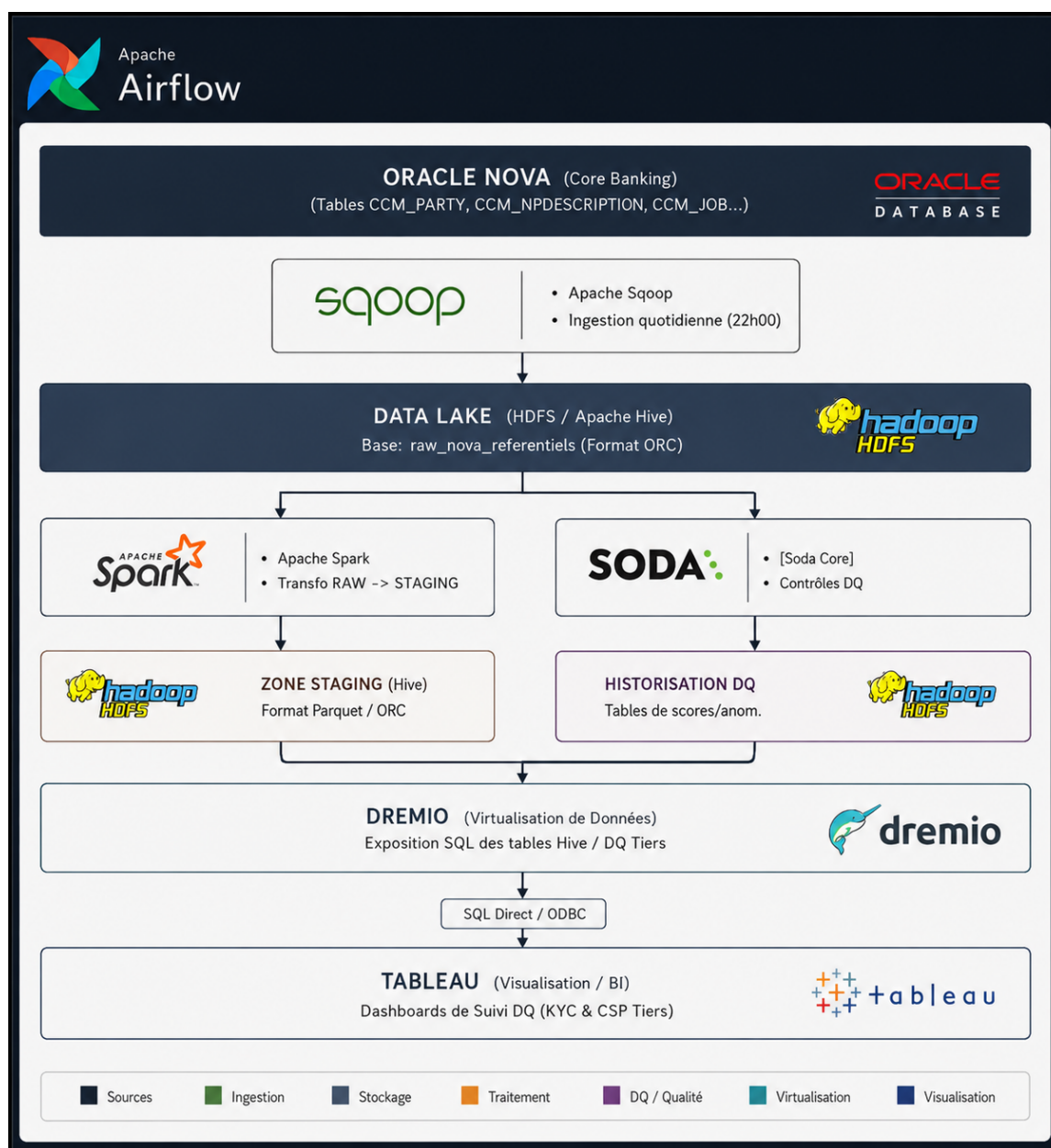


FIGURE 1.3 – Architecture du pipeline de données Référentiel Tiers PP Clients

Étape 1 — Système opérant : Oracle NOVA

La source de données est le système de *Core Banking* **Oracle NOVA**, qui héberge le schéma **REFERENTIELTIERS**. Les tables sources principales utilisées sont :

- **CCM_PARTY** : informations générales du tiers (identifiant, numéro client, marché, état)
- **CCM_NATURALPERSON** : lien entre le tiers et sa description de personne physique
- **CCM_NPDESCRIPTION** : données d'état civil et KYC (nom, prénom, date de naissance, nationalité, FATCA)
- **CCM_JOB** : données socio-professionnelles CSP (secteur, profession, CSP, employeur)
- **CCM_PORTFOLIO** : portefeuille commercial (gestionnaire associé)
- **CCM_CONTEXT** : contexte de création du dossier client (agence, utilisateur)

Étape 2 — Ingestion RAW vers HDFS/Hive

Le transfert quotidien des données depuis Oracle NOVA vers le Data Lake est orchestré par **Apache Airflow**. Le DAG principal du domaine Tiers est `dailyow22`, qui s'exécute chaque soir à **22h00**. Il utilise **Apache Sqoop** en mode *OverWrite* (OW) Full pour importer l'ensemble des tables `CCM_*` vers la base Hive `novatiers`, dans le répertoire HDFS `/data/raw/novatiers/`, au format ORC.

Un second DAG, `nova_referentieltiers_clients`, alimente en parallèle la base Hive `raw_nova_referentieltiers`, qui est la base **directement consommée par Dremio** pour la construction des vues analytiques.

Étape 3 — Transformation STAGING

Un job **Apache Spark** (DAG `referentieltiers_ingestion`, script `staging_novareferentieltiers`) transforme et charge les données depuis la zone RAW vers la zone STAGING, dans la base Hive `staging_nova_referentieltiers`, au format Parquet/ORC.

Étape 4 — Virtualisation Dremio

Dremio expose les données de staging via **10 vues virtuelles** dans le schéma `staging-nova-refere`

- `v.Customers` (source : `CCM_PARTY`)
- `v.natural_person_description` (source : `CCM_NPDESCRIPTION`)
- `v.customer_job` (source : `CCM_JOB`)
- `v.portfolio` (source : `CCM_PORTFOLIO`)
- `v.customer_creation_context` (source : `CCM_CONTEXT`)
- `v.customer_segmentation` et `v.customer_segmentation_members`
- `v.operational_posts` (source : schéma `staging-e_params`)

Étape 5 — Vue finale CIHONE (28 colonnes)

La vue analytique finale **CIHONE** résulte d'une jointure entre les 10 vues Dremio, filtrée sur `customer_type=1` et `code_marche IN (1,8)`, pour produire 28 colonnes décrivant chaque client PP actif de CIH Bank. Cette vue alimente les outils de reporting (Tableau) et les analyses métiers.

1.2.4 Critique de l'existant

L'approche décrite précédemment présente plusieurs limites majeures qui entravent la valorisation et la fiabilité du patrimoine de données de la banque :

- **Approche exclusivement réactive** : Le système de gestion attend que les erreurs atteignent les utilisateurs finaux (les métiers) avant d'être traitées, ce qui impacte négativement les processus décisionnels et génère de la frustration.
- **Lourdeur des processus de correction** : La résolution d'une simple anomalie nécessite la mobilisation asynchrone de plusieurs intervenants (métiers, Data Project, Data Engineers, équipes SI) pour des tâches de réconciliation manuelles extrêmement chronophages.
- **Risques liés aux données critiques** : L'absence de vérification en amont, particulièrement pour des données réglementaires et stratégiques comme le KYC (Know Your Customer) et les CSP, expose la banque à des risques de non-conformité et fausse les modèles analytiques.

Ces limites soulignent l'incapacité de ce fonctionnement réactif à répondre de manière optimale à la volumétrie croissante des données et aux exigences réglementaires.

1.2.5 Analyse de la problématique technique

Absence de contrôle qualité automatisé sur le pipeline

Bien que ce pipeline de données soit robuste et opérationnel, il ne dispose d'aucun mécanisme automatisé de contrôle de la qualité des données. Les données transitent quotidiennement depuis Oracle NOVA vers Dremio **sans aucune validation intermédiaire** sur les colonnes critiques KYC et CSP.

Cette absence de contrôle génère plusieurs problèmes identifiés dans le domaine Tiers :

- **Incohérences genre / civilité** : des clients de genre masculin associés à une civilité féminine (MME, MLLE) ou inversement, signalant une saisie erronée dans NOVA.
- **Données CSP massivement nulles ou parasites** : les colonnes `professional_sector`, `occupation` et `socioprofessional_category` présentent des taux de valeurs nulles significatifs. Le champ `employer` contient de nombreuses valeurs parasites non normalisées telles que « AUTRE », « LUI MEME » ou « BIEN VU ».
- **Dates de naissance approximatives** : une proportion notable de clients enregistre une date de naissance au 1er janvier, signalant une saisie par défaut faute d'information précise.
- **Non-conformité FATCA** : des clients de nationalité américaine ne disposent d'aucun identifiant FATCA, exposant la banque à un risque réglementaire critique.
- **Risque de fraîcheur** : aucune alerte ne se déclenche si le batch `dailyow22` échoue ou se termine en retard, laissant Dremio exposer des données potentiellement obsolètes.

Absence d'historisation et de traçabilité

Les anomalies détectées manuellement ne sont ni tracées ni historisées. Il est donc impossible de mesurer l'évolution de la qualité des données dans le temps, d'identifier des tendances dégradantes ou de prouver une amélioration après une action corrective menée par les Data Stewards dans NOVA.

Absence d'outillage pour les Data Stewards

Les **Data Stewards** et **Data Owners** du domaine Tiers n'ont pas accès à un outil centralisé leur permettant de visualiser les anomalies, de les prioriser par criticité et de suivre l'avancement des corrections. La remédiation est aujourd'hui réactive et non structurée.

1.2.6 Solution proposée

Pour pallier les lacunes du système existant, il s'avère indispensable d'amorcer une transition d'un fonctionnement réactif vers une gestion proactive de la qualité des données. La solution que nous proposons consiste à concevoir et à mettre en œuvre un **pipeline automatisé de contrôle de la qualité des données**, ciblant dans un premier temps les données critiques liées au KYC et aux CSP.

Cette solution permettra notamment de :

- **Anticiper et prévenir les anomalies** : Les contrôles s'exécuteront automatiquement dès l'ingestion ou la transformation des données, bien avant qu'elles ne soient mises à la disposition des utilisateurs finaux.

- **Automatiser les vérifications** : Le pipeline appliquera systématiquement un ensemble de règles métiers et techniques prédéfinies pour isoler les doublons, les valeurs manquantes ou les incohérences structurelles.
- **Mettre en place une supervision continue** : Les résultats de ces contrôles seront consolidés dans des tableaux de bord, offrant aux équipes Data et Métiers une visibilité claire et en temps réel sur la santé des données.

Ce changement de paradigme garantira un gain de temps considérable, fiabilisera les analyses et renforcera la gouvernance globale de la donnée au sein de CIH BANK.

1.2.7 Solution proposée : automatisation de la Data Quality

Vue d'ensemble de la solution

Pour répondre à ces problématiques, nous proposons de concevoir et de mettre en œuvre un **pipeline automatisé de Data Quality (DQ)** déclenché quotidiennement après chaque exécution du DAG `dailyow22`, basé sur la bibliothèque Python **Soda Core**. La solution s'articule autour de quatre composantes majeures :

1. **Moteur de contrôle DQ (Soda Core)** : un script Python exécutant des règles de qualité paramétrables sur les tables Hive des zones RAW et STAGING du domaine Tiers.
2. **Couche de persistance (HDFS/Hive)** : des tables Hive dédiées pour historiser quotidiennement les scores DQ et le détail des anomalies détectées.
3. **Exposition analytique (Dremio)** : des vues virtuelles Dremio exposant les résultats DQ historisés pour les outils de consommation.
4. **Dashboard de suivi (Tableau)** : un tableau de bord interactif permettant aux Data Stewards et Data Owners de visualiser les KPIs de qualité, l'évolution temporelle des scores et la liste priorisée des anomalies à corriger.

Soda Core : bibliothèque Python de Data Quality

Soda Core est une bibliothèque Python open source dédiée au contrôle de la qualité des données. Elle permet de définir des règles de qualité (appelées *checks*) dans des fichiers de configuration YAML, de les exécuter sur différentes sources de données (Hive, Spark, PostgreSQL, Snowflake, etc.) et de collecter les résultats de manière programmatique. [REF]

Les avantages de Soda Core pour notre contexte sont les suivants :

- **Compatibilité Hive / Spark** : Soda Core dispose d'un connecteur natif compatible avec l'environnement Hadoop de CIH Bank (`soda-core-spark`).
- **Configuration déclarative** : les règles DQ sont définies en YAML, ce qui facilite leur maintenance et leur évolution sans modifier le code Python.
- **Intégration Airflow** : Soda Core s'intègre nativement dans les DAGs Airflow via un opérateur Python, permettant de le déclencher automatiquement après chaque ingestion.
- **Résultats programmatiques** : les résultats des checks (valeur calculée, seuil, statut) sont accessibles en Python, ce qui permet de les persister facilement dans des tables Hive.

Périmètre des contrôles : la zone RAW uniquement

Le choix de cibler la **zone RAW** du Data Lake comme unique point de contrôle est délibéré et justifié par plusieurs raisons :

- **Fidélité à la source** : la zone RAW (base Hive `raw_nova_referentietiers`, chemin HDFS `/data/raw/novatiers/`) est la copie exacte et non transformée des tables `CCM_*` issues d'Oracle NOVA. Contrôler la qualité sur RAW revient à mesurer directement la qualité du système opérant.
- **Isolation des responsabilités** : en contrôlant avant la transformation STAGING, on distingue clairement les anomalies issues du système source (responsabilité des Data Stewards Tiers) des éventuels artefacts introduits par les scripts de transformation.
- **Remédiation à la source** : les anomalies détectées sur RAW peuvent être remontées directement aux équipes saisie des agences pour correction dans NOVA, sans ambiguïté sur leur origine.

Déclenchement automatique post-ingestion

La solution s'insère dans le workflow Airflow existant. Une **tâche Python dédiée** (`run_dq_tiers_checks`) est ajoutée dans le DAG `dailyow22`, en dépendance de toutes les tâches d'ingestion Sqoop des tables `CCM_*`. Ainsi, les contrôles DQ se lancent **automatiquement chaque soir à 22h**, dès que les données RAW sont disponibles, sans intervention manuelle.

Historisation et traçabilité quotidienne

Les résultats de chaque exécution sont persistés dans deux tables Hive partitionnées par date (`run_date`) dans une base dédiée `dq_results` :

- `dq_tiers_scores` : enregistre, pour chaque combinaison table/colonne/règle, la valeur calculée, le seuil configuré, le statut (OK/KO) et le score DQ normalisé sur 100.
- `dq_tiers_anomalies` : enregistre le détail des enregistrements clients (`identifier`, `radical`) en situation d'anomalie, avec la description de la règle violée et la criticité associée.

Cette historisation permet de mesurer l'**évolution temporelle** de la qualité des données à la source et de quantifier l'impact des corrections effectuées dans NOVA par les Data Stewards.

1.3 Choix du modèle de développement

Dans le cadre de ce projet de fin d'études, nous avons opté pour une approche de développement classique et séquentielle, s'apparentant au modèle en cascade (Waterfall) ou au cycle en V.

Ce choix a été motivé par la nature du projet de conception d'un pipeline de contrôle qualité, qui nécessite des étapes bien définies et successives : une étude approfondie de l'existant, une modélisation et une conception précises des règles de qualité, suivies d'une phase de développement (implémentation du pipeline) et enfin d'une période de tests et de validation.

Bien que ce modèle classique impose un rythme séquentiel et que l'avancement puisse parfois connaître des fluctuations (en fonction des contraintes techniques, de la disponibilité des données ou des dépendances avec d'autres équipes), il permet de garantir un cadre de travail structuré. Il assure notamment une validation claire et une documentation rigoureuse de chaque jalon avant de passer à l'étape suivante. Cette structuration s'est avérée particulièrement pertinente pour s'assurer que les exigences métiers, notamment

réglementaires concernant les données KYC et CSP, soient parfaitement intégrées dès la phase de conception.

1.4 Planning prévisionnel

Afin de mener à bien ce projet de fin d'études au sein de l'entité Data Management de CIH BANK, nous avons défini un planning prévisionnel rigoureux s'étalant sur une durée de six mois, du 9 février au 9 août 2026.

Le déroulement du projet a été structuré autour de quatre grandes phases séquentielles, conformément au cycle de vie adopté :

- **Étude de l'existant et cadrage** : Imprégnation du contexte bancaire, compréhension des limites des processus de contrôle qualité actuels et recueil des besoins.
- **Conception** : Modélisation des règles de qualité de données (Data Quality) spécifiques au KYC et aux CSP, et conception de l'architecture du pipeline.
- **Réalisation** : Phase centrale dédiée à l'implémentation technique des règles métiers et du pipeline automatisé.
- **Tests et Déploiement** : Validation de la fiabilité des contrôles, tests de bout en bout, et élaboration des tableaux de bord de supervision.

La figure 1.4 ci-dessous illustre le diagramme de Gantt représentant l'échelonnement temporel de ces différentes étapes tout au long du stage.

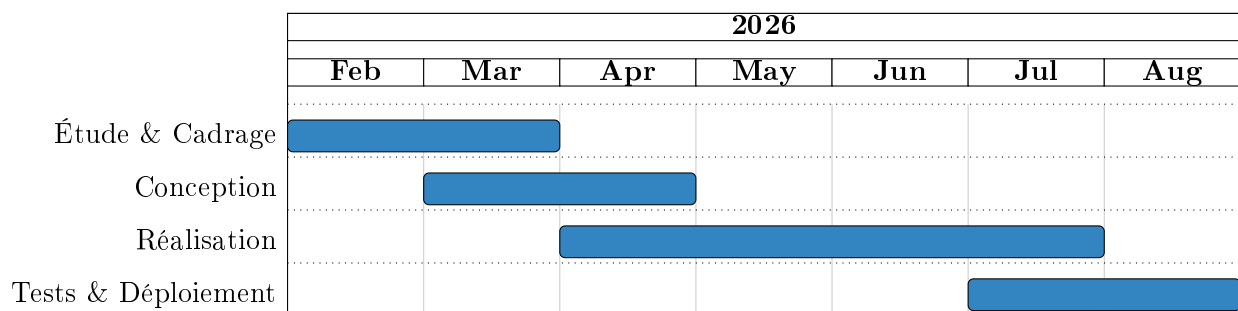


FIGURE 1.4 – Diagramme de Gantt du planning prévisionnel du projet

Conclusion

Pour conclure, CIH BANK est une institution bancaire marocaine majeure, fortement structurée et engagée dans la valorisation de la donnée à travers son Data Office. L'étude de l'existant a permis de mettre en évidence les processus actuels de gestion des données et leurs limites, ce qui justifie la nécessité d'un pipeline automatisé de contrôle qualité. Ces éléments constituent la base de notre projet et nous permettent d'entamer sereinement la phase de spécification des besoins dans le chapitre suivant.

Chapitre 2

Analyse des Besoins et Spécification

Introduction

Après avoir décrit le cadre général du projet et analysé le fonctionnement de la solution existante au sein de CIH BANK, ce chapitre se focalise sur la définition claire et rigoureuse des besoins. Nous y détaillons d’une part les besoins fonctionnels, c’est-à-dire les fonctionnalités attendues du pipeline de Data Quality, et d’autre part les besoins non fonctionnels, liés aux exigences techniques, de performance et de sécurité. Enfin, nous présentons la modélisation fonctionnelle à travers la spécification des cas d’utilisation (Use Cases) et de leurs acteurs, conformément aux standards académiques.

2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les actions que le système doit être capable de réaliser pour répondre aux problématiques de qualité des données identifiées sur le Référentiel Tiers. Ils se répartissent en quatre fonctionnalités majeures :

2.1.1 Automatisation et planification des contrôles

- **Déclenchement automatique** : Le système doit lancer l’évaluation de la qualité des données de manière planifiée, immédiatement après l’exécution complète du batch quotidien d’ingestion des tables sources Oracle NOVA (DAG Airflow `dailyow22` s’exécutant chaque soir à 22h00).
- **Orchestration intégrée** : L’exécution du scan de qualité doit être orchestrée au sein d’Apache Airflow, permettant de suivre son statut d’exécution (Succès/Échec) et de consulter les logs en temps réel.

2.1.2 Configuration déclarative des règles de qualité

- **Moteur de règles dynamique** : Le système doit permettre de définir les contrôles de qualité (Missing, Invalid, Freshness, Consistency, Uniqueness) dans des fichiers de règles déclaratifs séparés du code source.
- **Graduation des seuils et des alertes** : Pour chaque contrôle, le système doit pouvoir évaluer des seuils paramétrables (ex : taux de valeurs manquantes inférieur à 5%) et générer un statut de warning ou d’error selon la gravité de l’écart.
- **Gestion des métadonnées** : Chaque règle doit être documentée avec des attributs métier (nom de la règle, dimension DQ associée, domaine KYC ou CSP, criticité, etc.).

2.1.3 Historisation et traçabilité des indicateurs

- **Persistance journalière** : À chaque exécution, le système doit sauvegarder les métriques calculées et les résultats de chaque contrôle de qualité dans une table d'historique de scores (base Hive dédiée `dq_results`), indexée par date d'exécution.
- **Identification des anomalies (Remédiation)** : Le système doit capturer de manière détaillée la liste des enregistrements clients en anomalie en stockant leur identifiant technique (UUID) et leur radical (numéro client), ainsi que le libellé du contrôle en échec et sa criticité. Ces informations doivent être stockées dans une table d'anomalies pour actions correctives.

2.1.4 Restitution visuelle et aide à la décision

- **Exposition des données** : Les résultats d'historisation de la qualité doivent être exposés de manière transparente sous forme de vues SQL pour les outils décisionnels.
- **Suivi de l'évolution temporelle** : Le tableau de bord doit permettre de suivre la tendance d'évolution des scores DQ dans le temps (30 derniers jours).
- **Priorisation des remédiations** : Le tableau de bord doit fournir une liste opérationnelle des clients en anomalie triée par criticité décroissante, exportable par les Data Stewards pour traitement correctif dans le système source NOVA.

2.2 Besoins non fonctionnels

Les besoins non fonctionnels expriment les exigences qualitatives, opérationnelles et techniques que la solution doit respecter pour être viable et performante dans l'environnement de production de CIH BANK :

2.2.1 Performance et passage à l'échelle (Scalabilité)

Le Référentiel Tiers de la banque regroupe **plusieurs millions** de dossiers clients (personnes physiques). Les règles de cohérence inter-colonnes ou de doublons nécessitent des jointures lourdes. Le moteur DQ doit être capable d'interroger et d'évaluer ce volume massif de données en un temps minimal (moins de 15 minutes) sans dégrader les ressources du Data Lake. Cela justifie l'utilisation d'un moteur de calcul distribué (Apache Spark) pour le traitement.

2.2.2 Robustesse et découplage (Tolérance aux pannes)

L'exécution des contrôles de qualité ne doit pas bloquer la disponibilité des données pour les outils de reporting en cas d'anomalie. Si un seuil de qualité est franchi ou si le scan échoue, le pipeline d'exposition (Dremio/Tableau) doit continuer à fonctionner en mode dégradé en signalant l'alerte. De plus, la tâche DQ doit s'exécuter même si certaines tables d'ingestion n'ont été chargées que partiellement.

2.2.3 Maintenabilité et extensibilité

L'ajout de nouvelles règles ou l'intégration de nouvelles colonnes du Référentiel Tiers (par exemple pour de nouveaux critères réglementaires ou pour les personnes morales) doit se faire sans modification du code Python principal de la solution. Les fichiers de

configuration YAML et la structure des tables Hive de résultats doivent être conçus de manière générique pour supporter l’extensibilité.

2.2.4 Sécurité et gouvernance

Les détails d’anomalies de qualité des données contiennent des informations clients sensibles (noms, prénoms, radicaux). Seuls les collaborateurs habilités du Data Office (Data Stewards, Data Project) et les Data Owners des départements concernés doivent être autorisés à accéder aux détails des anomalies. La solution doit ainsi s’appuyer sur la gestion des droits d’accès existante de Dremio et de Tableau.

2.3 Modélisation des cas d’utilisation

Afin de représenter formellement les interactions entre les utilisateurs et la solution de Data Quality, nous présentons ci-dessous les acteurs identifiés ainsi que la modélisation globale des cas d’utilisation.

2.3.1 Acteurs du système

Nous identifions trois acteurs principaux interagissant avec le système :

- **Data Steward / Data Owner (Acteur humain)** : Il s’agit des gestionnaires de la donnée. Ils définissent les règles de qualité dans les fichiers YAML, consultent les dashboards Tableau pour suivre les indicateurs globaux, et exportent les anomalies pour lancer des chantiers de correction.
- **Système d’Orchestration Airflow (Acteur système)** : Il s’agit du planificateur automatique. Il déclenche le pipeline de contrôles chaque soir après l’ingestion et remonte le statut d’exécution.
- **Système Source (Oracle NOVA) (Acteur externe)** : C’est le récepteur final des corrections de données opérées manuellement par les gestionnaires ou les agences suite aux alertes DQ.

2.3.2 Diagramme global des cas d’utilisation

Le diagramme ci-dessous synthétise graphiquement les cas d’utilisation de la solution et leurs acteurs associés.

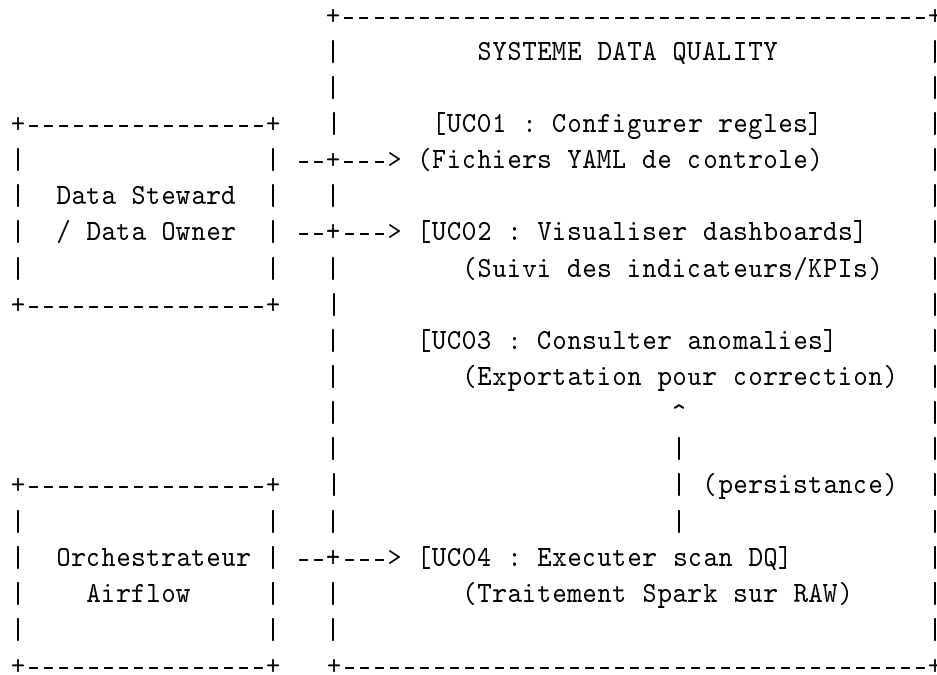


FIGURE 2.1 – Diagramme global des cas d'utilisation de la solution DQ

2.3.3 Description textuelle des cas d'utilisation principaux

Nous détaillons ci-dessous le fonctionnement interne des trois cas d'utilisation principaux de notre système :

Cas d'utilisation 01 : Configurer les règles de qualité

- **Acteur principal** : Data Steward.
- **Préconditions** : Accès aux spécifications métier (critères KYC et CSP).
- **Scénario nominal** :
 1. Le Data Steward ouvre ou crée un fichier de configuration YAML (`checks_kyc.yml` ou `checks_csp.yml`).
 2. Il y déclare les contrôles pour chaque colonne concernée (ex : `missing_count`, `valid values`).
 3. Il associe à chaque règle ses attributs métier (criticité, domaine, dimension).
 4. Il sauvegarde le fichier et le déploie sur le serveur d'exécution.
- **Postconditions** : Les nouvelles règles sont prêtes à être exécutées lors du prochain scan automatique.

Cas d'utilisation 02 : Exécuter le scan de qualité des données

- **Acteur principal** : Orchestrateur Airflow (Système).
- **Préconditions** : Ingestion dailyow22 terminée, fichiers YAML configurés.
- **Scénario nominal** :
 1. Airflow déclenche la tâche `run_dq_tiers_checks` suite à la fin du chargement Sqoop.
 2. Le moteur DQ initialise une `SparkSession` et charge les règles YAML.
 3. Le moteur applique les contrôles sur la zone RAW Hive de manière distribuée.
 4. Les anomalies et scores calculés sont stockés dans la base `dq_results` de Hive.

5. Airflow marque la tâche comme terminée et envoie les logs d'exécution.
- **Postconditions** : Les données de qualité sont prêtes à être consultées dans Dremio.

Cas d'utilisation 03 : Consulter les anomalies et remédier

- **Acteurs** : Data Steward, Data Owner.
- **Préconditions** : Exécution du scan DQ finalisée avec succès.
- **Scénario nominal** :
 1. Le Data Steward se connecte au dashboard Tableau de qualité des données.
 2. Il consulte l'onglet « Anomalies et Remédiation » pour voir la liste des enregistrements non conformes.
 3. Il filtre les anomalies par criticité (ex : CRITIQUE) et par domaine (ex : KYC).
 4. Il exporte la liste contenant les radicaux clients concernés.
 5. Il transmet la liste aux agences pour correction des données erronées directement dans Oracle NOVA.
- **Postconditions** : Les données corrigées dans NOVA seront ré-ingérées le lendemain soir à 22h, et le score DQ s'améliorera automatiquement.

Conclusion

Ce chapitre a permis de formaliser les besoins fonctionnels et non fonctionnels, définissant de manière précise les objectifs techniques et opérationnels du pipeline de qualité des données. La modélisation des acteurs et des cas d'utilisation à l'aide d'UML (Usecases) clarifie les frontières et le comportement de notre système. Fort de ces spécifications, le chapitre suivant présente la conception globale et détaillée de la solution.

Chapitre 3

Conception de la Solution

Introduction

Ce chapitre présente la conception détaillée du pipeline de Data Quality sur le Référentiel Tiers de CIH Bank. Les contrôles de qualité s'appliquent **exclusivement à la zone RAW** du Data Lake, qui constitue la copie exacte et fidèle des tables CCM_* issues du système opérant Oracle NOVA. Nous définirons l'architecture globale de la solution, le modèle de données pour la persistance des résultats DQ, le catalogue complet des règles de contrôle, ainsi que la conception des composants d'orchestration, d'exposition et de visualisation.

3.1 Architecture globale de la solution

3.1.1 Vue d'ensemble

La solution de Data Quality s'insère dans le pipeline existant du domaine Tiers comme une **couche transversale de contrôle** ciblant exclusivement la **zone RAW**. La logique est simple : la zone RAW est la copie conforme du système opérant Oracle NOVA. Y détecter des anomalies équivaut à mesurer la qualité réelle des données à la source, avant toute transformation. Le schéma ci-dessous illustre l'architecture globale de bout en bout.

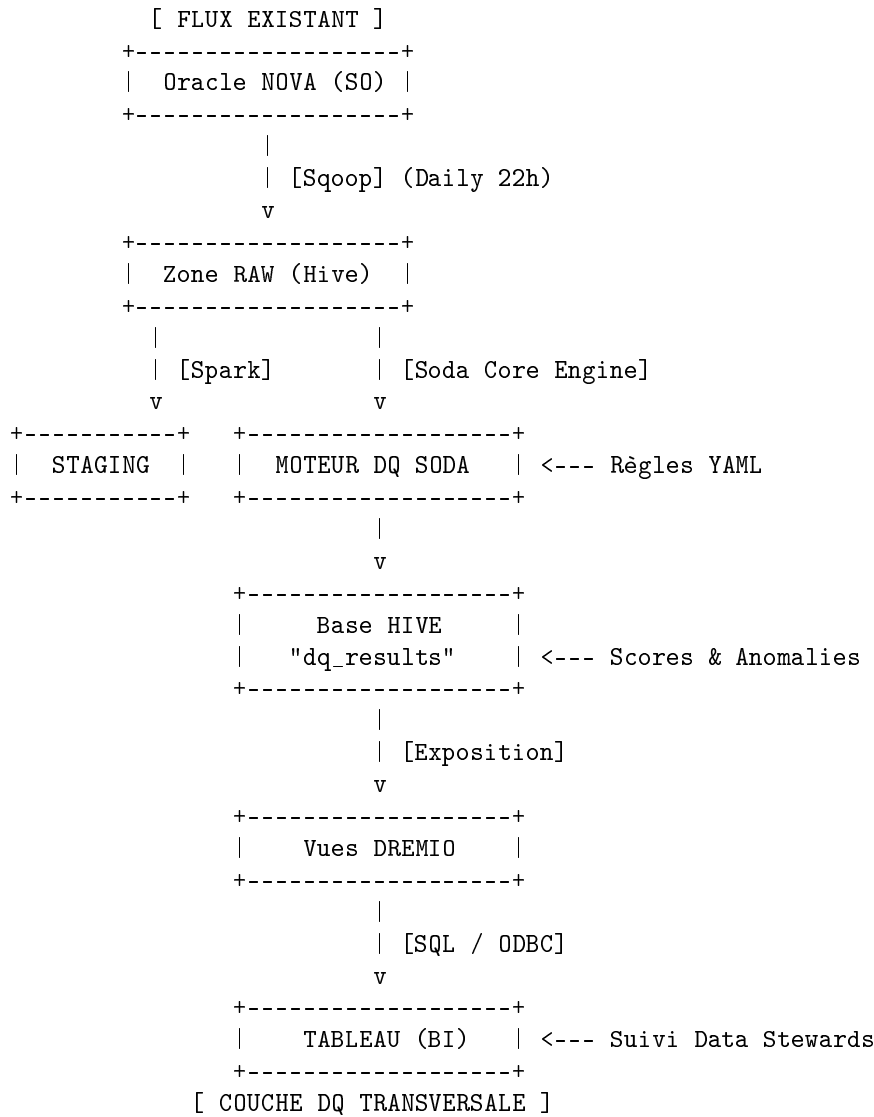


FIGURE 3.1 – Architecture globale de la solution de Data Quality

La solution comprend **cinq composantes principales** :

1. **Source** : Oracle NOVA (Core Banking) — inchangée
2. **Ingestion RAW** : DAG Airflow dailyow22 (22h) — inchangée
3. **Moteur DQ** : Script Python Soda Core (`dq_tiers_checks.py`) exécuté **sur la zone RAW** — **nouveau**
4. **Persistence** : Tables Hive de résultats DQ (base `dq_results`) — **nouveau**
5. **Exposition & Visualisation** : Vues Dremio `dq-tiers` + Dashboard Tableau — **nouveau**

3.1.2 Outils et technologies

Le tableau 3.1 présente les outils et technologies retenus pour la mise en œuvre de la solution.

Outil	Rôle dans la solution	Justification
Oracle NOVA	Système opérant source	Core Banking CIH Bank
Apache Sqoop	Ingestion Oracle → HDFS	Pipeline existant
Apache Airflow	Orchestration des DAGs	Pipeline existant — ajout d'une tâche DQ
Apache Hive / HDFS	Stockage RAW et STAGING	Data Lake CIH Bank
Apache Spark	Transformation STAGING	Pipeline existant
Soda Core (Python)	Moteur de contrôle DQ	Open source, compatible Spark/Hive, config YAML
Dremio	Virtualisation et exposition SQL	Data Virtualization existante
Tableau	Dashboard de visualisation DQ	Outil de reporting standard CIH Bank

TABLE 3.1 – Outils et technologies de la solution DQ Tiers



Oracle NOVA — Système opérant (Core Banking)

Système source des données du Référentiel Tiers. Héberge les tables CCM_PARTY, CCM_NPDDESCRIPTION, CCM_JOB contenant les informations KYC et CSP des clients PP.



Apache Sqoop — Ingestion batch Oracle vers HDFS

Outil de transfert massif de données entre bases relationnelles et Hadoop. Utilisé en mode *OverWrite* (OW) Full pour charger quotidiennement les tables CCM_* vers la zone RAW Hive `raw_nova_referentieltiers`.



Apache Airflow — Orchestration des pipelines de données

Plateforme d'orchestration de workflows. Le DAG `dailyow22` coordonne les tâches d'ingestion Sqoop chaque soir à 22h00. La tâche DQ Soda Core est ajoutée à ce DAG en aval de l'ingestion.



HDFS / Apache Hive — Data Lake CIH Bank

Système de stockage distribué (HDFS) et moteur de requêtage SQL (Hive). Les données RAW sont stockées dans `/data/raw/novatiers/` au format ORC. C'est sur ces tables RAW que les contrôles Soda Core sont exécutés.



Apache Spark — Moteur de transformation RAW vers STAGING

Moteur de traitement distribué en mémoire. Transforme et charge les données de la zone RAW vers la zone STAGING (`staging_nova_referentieltiers`). Mobilisé également par Soda Core via le connecteur `soda-core-spark`.



Soda Core — Moteur de Data Quality (Python)

Bibliothèque Python open source de contrôle de la qualité des données. Les règles DQ sont définies en YAML (`checks_kyc.yml`, `checks_csp.yml`) et exécutées sur les tables RAW Hive. Les résultats sont persistés dans Hive pour historisation quotidienne.



Dremio — Virtualisation et exposition SQL des résultats DQ

Plateforme de virtualisation de données exposant les tables Hive sous forme de vues SQL. Les scores et anomalies DQ sont exposés dans le schéma `dq-tiers` et consommés directement par Tableau sans déplacement de données.



Tableau — Dashboard de suivi de la qualité des données

Outil de visualisation connecté à Dremio via ODBC. Permet aux Data Stewards et Data Owners du domaine Tiers de suivre l'évolution des scores DQ KYC/CSP, de visualiser les anomalies et de prioriser les corrections dans le système NOVA.

3.2 Catalogue des règles de Data Quality

3.2.1 Principes de conception des règles

Les règles de Data Quality ont été définies selon les principes suivants :

- **Fondement métier** : chaque règle est justifiée par un besoin métier concret (conformité réglementaire KYC, fiabilité du scoring CSP, détection d'anomalies de saisie).
- **Périmètre RAW** : toutes les règles s'appliquent sur les tables de la **zone RAW Hive** (base `raw_nova_referentieltiers`) qui sont la copie conforme des tables `CCM_*` d'Oracle NOVA.
- **Seuils quantifiés** : chaque règle dispose d'un seuil d'alerte mesurable (ex : taux de valeurs nulles $> 5\%$ = alerte).
- **Criticité graduée** : chaque règle est classée selon quatre niveaux de criticité : CRITIQUE, HAUTE, MOYENNE, FAIBLE.
- **Action corrective définie** : chaque règle est associée à une action corrective à destination des Data Stewards pour correction dans NOVA.

3.2.2 Règles DQ sur les colonnes KYC

Les règles KYC portent sur les données d'état civil et de conformité réglementaire. La table RAW cible est `raw_nova_referentieltiers.ccm_npdescription`.

TABLE 3.2 – Règles DQ — Colonnes KYC

Colonne(s)	Dimension	Règle / Seuil	Criticité	Action corrective
name	Complétude	0 valeur nulle tolérée	CRITIQUE	Blocage saisie NOVA — audit dossiers

Colonne(s)	Dimension	Règle / Seuil	Criticité	Action corrective
first_name	Complétude	0 valeur nulle tolérée	CRITIQUE	Blocage saisie NOVA — audit dossiers
birthdate	Complétude	0 valeur nulle tolérée	CRITIQUE	Compléter via NOVA avec pièce d'identité
birthdate	Validité	Taux dates au 01/01 < 15%	HAUTE	Flagger date_naissance_approx — ne pas utiliser pour le scoring
civility	Validité	Valeurs IN (M., MME, MLLE) uniquement	HAUTE	Corriger valeurs hors référentiel dans NOVA
gender	Validité	Valeurs IN (M, F) uniquement	HAUTE	Corriger valeurs hors référentiel dans NOVA
gender + civility	Cohérence	Genre M ≠ MME/MLLE ; Genre F ≠ M.	CRITIQUE	Audit croisé — correction manuelle dans NOVA
nationality_country	Complétude	0 valeur nulle tolérée	HAUTE	Compléter via NOVA avec pièce d'identité
fatca_identification_number	Conformité	Clients US ou double-nationalité US : 0 FATCA null	CRITIQUE	Remonter immédiatement à la Conformité CIH
birth_city	Validité	100% des codes résolus dans le référentiel villes	MOYENNE	Mapper codes numériques vers libellés via table référentiel villes BAM/CIH

3.2.3 Règles DQ sur les colonnes CSP

Les règles CSP portent sur les données socio-professionnelles. La table RAW cible est `raw_nova_referentieltiers.ccm_job`.

TABLE 3.3 – Règles DQ — Colonnes CSP

Colonne(s)	Dimension	Règle / Seuil	Criticité	Action corrective
professional_sector	Complétude	Taux NULL < 5%	HAUTE	Compléter via NOVA — agences prioritaires
occupation	Complétude	Taux NULL < 5%	HAUTE	Compléter via NOVA
socioprofessional_category	Complétude	Taux NULL < 5%	HAUTE	Compléter via NOVA
professional_situation	Validité	Valeurs IN (SAL, FON, TRI, AUT, EMP, NDE)	HAUTE	Corriger valeurs hors référentiel
employer	Validité	Taux valeurs parasites (AUTRE, LUI MEME...) < 30%	HAUTE	Normaliser : mapper 'LUI MEME' → TRI, 'AUTRE' → NULL
Triple CSP	Cohérence	Secteur + Profession + CSP non contradictoires	HAUTE	Vérifier cohérence et corriger dans NOVA

Colonne(s)	Dimension	Règle / Seuil	Criticité	Action corrective
ordnum (multi-emplois)	Unicité	Doublons avec main=0 documentés	MOYENNE	Sélectionner emploi principal (main=1) dans la jointure CI-HONE
politically_exposed	Complétude	0 valeur nulle tolérée	CRITIQUE	Demander déclaration PPE à l'agence

3.2.4 Règles DQ Pipeline (Fraîcheur)

Colonne(s)	Dimension	Règle / Seuil	Criticité	Action corrective
batch_datetime_eaac4	Fraîcheur	Retard < 26h depuis dernière exécution	HAUTE	Vérifier DAG Airflow et logs Sqoop
code_segment	Cohérence	0 client sans segment valide	HAUTE	Assigner segment par défaut selon code_marche

TABLE 3.4 – Règles DQ — Fraîcheur et Pipeline

3.3 Modèle de données DQ (Persistance)

3.3.1 Base Hive dq_results

Nous créons une base Hive dédiée `dq_results` pour stocker les résultats des contrôles DQ. Elle est distincte des bases métier (`novatiers`, `staging_nova_referentieltiers`) pour garantir la séparation des responsabilités.

3.3.2 Table dq_tiers_scores — Historique des scores

Cette table enregistre, pour chaque exécution quotidienne, le résultat agrégé de chaque règle de contrôle.

Colonne	Type	Description
run_date	DATE (partition)	Date d'exécution du contrôle DQ
dag_name	STRING	Nom du DAG déclencheur (dailyow22)
table_name	STRING	Table Hive contrôlée
column_name	STRING	Colonne contrôlée
check_name	STRING	Nom descriptif de la règle DQ
dimension	STRING	Dimension DQ : completeness, validity, consistency, freshness
criticite	STRING	CRITIQUE / HAUTE / MOYENNE / FAIBLE
domaine	STRING	KYC / CSP / PIPELINE
metric_value	DOUBLE	Valeur calculée (ex : 0.023 = 2,3% de nulls)
threshold	DOUBLE	Seuil d'alerte configuré
passed	BOOLEAN	TRUE si le check est passé
score_dq	DOUBLE	Score normalisé 0–100 pour ce check

TABLE 3.5 – Modèle de la table `dq_results.dq_tiers_scores`

3.3.3 Table `dq_tiers_anomalies` — Détail des anomalies

Cette table enregistre le détail de chaque enregistrement client en situation d'anomalie, pour permettre aux Data Stewards de les identifier et de les corriger dans NOVA.

Colonne	Type	Description
run_date	DATE (partition)	Date d'exécution
table_name	STRING	Table source de l'anomalie
column_name	STRING	Colonne en anomalie
check_name	STRING	Règle DQ violée
identifiant	STRING	UUID du tiers en anomalie
radical	STRING	Numéro client CIH (clé bancaire)
anomaly_detail	STRING	Description de l'anomalie (ex : genre=M, civilite=MME)
criticite	STRING	Criticité de la règle

TABLE 3.6 – Modèle de la table `dq_results.dq_tiers_anomalies`

3.3.4 Partitionnement et format de stockage

Les deux tables sont **partitionnées par run_date** (DATE), ce qui permet :

- Une interrogation performante par période (dernier jour, 7 derniers jours, etc.)
- Une gestion du cycle de vie des données (rétention configurable)
- Un rechargement partiel en cas d'erreur sans affecter l'historique

Le format de stockage retenu est **ORC** (Optimized Row Columnar), cohérent avec les autres tables du Data Lake CIH Bank, pour une compression et des performances de requêtage optimales.

3.4 Conception du moteur DQ — Soda Core

3.4.1 Architecture du script `dq_tiers_checks.py`

Le script Python principal est organisé en trois phases :

1. **Initialisation** : connexion à la source de données Hive via Soda Core, chargement des fichiers de configuration YAML des règles.
2. **Exécution des checks** : Soda Core exécute les règles définies dans les fichiers YAML sur les tables Hive cibles et collecte les résultats.
3. **Persistance** : les résultats sont formatés et écrits dans les tables `dq_tiers_scores` et `dq_tiers_anomalies` via une `SparkSession`.

3.4.2 Fichiers de configuration YAML

Les règles DQ sont externalisées dans des fichiers YAML séparés par domaine, ce qui facilite leur maintenance sans modifier le code Python. Nous prévoyons trois fichiers :

- `checks_kyc.yml` : règles sur les colonnes KYC (`CCM_NPDESCRIPTION`)
- `checks_csp.yml` : règles sur les colonnes CSP (`CCM_JOB`)
- `checks_pipeline.yml` : règles de fraîcheur et cohérence pipeline

3.5 Conception de l'orchestration Airflow

3.5.1 Intégration dans le DAG `dailyow22`

La tâche DQ s'insère dans le DAG `dailyow22` existant, en **aval** de toutes les tâches d'ingestion des tables `CCM_*`. Elle ne bloque pas le pipeline en cas d'erreur (mode `TriggerRule.ALL_DONE`) afin de garantir que les résultats DQ sont produits même si certaines tables sont partiellement chargées.

La figure 3.2 illustre le positionnement de la tâche DQ dans le graphe de dépendances du DAG.

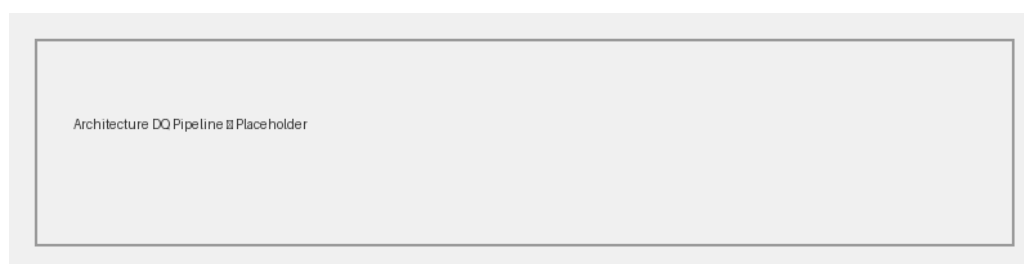


FIGURE 3.2 – Positionnement de la tâche DQ dans le DAG `dailyow22`

3.6 Conception des vues Dremio DQ

Un nouveau schéma Dremio `dq-tiers` est créé pour exposer les résultats DQ. Il comprend trois vues :

Vue Dremio	Source Hive	Usage
v.dq_scores_history	dq_tiers_scores	Historique des scores — courbes de tendance Tableau
v.dq_kpis_summary	dq_tiers_scores (agrégé)	KPIs globaux KYC/CSP par dimension et par jour
v.dq_anomalies_detail	dq_tiers_anomalies	Détail des clients en anomalie — liste pour Data Stewards

TABLE 3.7 – Vues Dremio du schéma dq-tiers

3.7 Conception du dashboard Tableau

Le dashboard Tableau « **Qualité Données Référentiel Tiers** » est conçu pour deux profils d'utilisateurs :

- **Data Owners et Direction** : vue exécutive avec scores globaux et alertes critiques
- **Data Stewards** : vue opérationnelle avec liste détaillée des anomalies à corriger

Le dashboard est structuré en **quatre onglets** :

1. **Vue Exécutive** : Score DQ global KYC + CSP du jour, évolution sur 30 jours, alertes critiques en rouge.
2. **Détail KYC** : Score par colonne KYC, heatmap par dimension DQ (Complétude / Validité / Cohérence), filtres par agence de rattachement.
3. **Détail CSP** : Score CSP, analyse des valeurs parasites dans **employer**, taux de nullité par colonne, tendance d'amélioration après corrections.
4. **Anomalies & Remédiation** : Liste paginée des tiers en anomalie, triée par criticité, exportable pour les Data Stewards. Lien direct vers le **radical** CIH pour investigation dans NOVA.

Conclusion

Ce chapitre a présenté la conception complète de la solution de Data Quality Tiers. Nous avons défini l'architecture globale à six composantes, le catalogue des règles DQ par domaine (KYC, CSP, Pipeline), le modèle de données de persistance dans Hive, la logique d'orchestration Airflow, les vues Dremio d'exposition et la structure du dashboard Tableau. Le chapitre suivant décrit la réalisation concrète de ces différents composants.

Chapitre 4

Réalisation de la Solution

Introduction

Ce chapitre présente la réalisation concrète du pipeline de Data Quality sur le Référentiel Tiers de CIH Bank. Tous les contrôles Soda Core s'effectuent exclusivement sur la **zone RAW** du Data Lake (base Hive `raw_nova_referentieltiers`), qui constitue la copie exacte et non transformée des tables `CCM_*` du système opérant Oracle NOVA. Nous détaillerons successivement l'environnement technique de développement, l'implémentation du script Soda Core, la création des tables Hive de persistance, l'intégration dans le DAG Airflow `dailyow22`, la mise en place des vues Dremio et la réalisation du dashboard Tableau.

4.1 Environnement technique de développement

4.1.1 Environnement Hadoop et cluster

Le développement et les tests ont été réalisés dans l'environnement Hadoop de CIH Bank. L'accès aux tables Hive et aux fichiers HDFS est effectué via les interfaces disponibles dans l'environnement Data Office, en utilisant les comptes de service existants du domaine Tiers.

Les versions des composants utilisés dans l'environnement de production sont les suivantes :

Composant	Version
Apache Hadoop (HDFS)	Cluster CIH Bank
Apache Hive	Cluster CIH Bank
Apache Spark	Cluster CIH Bank
Apache Airflow	Cluster CIH Bank
Dremio	Instance CIH Bank
Python	3.x
Soda Core	soda-core-spark
Tableau	Tableau Desktop / Server CIH Bank

TABLE 4.1 – Versions des composants de l'environnement technique

4.1.2 Installation de Soda Core

Soda Core est installé dans l'environnement Python accessible depuis les workers Airflow. Le connecteur retenu est `soda-core-spark`, qui permet à Soda Core d'utiliser une `SparkSession` existante comme moteur d'exécution pour interroger les tables Hive de la zone RAW.

```
1 pip install soda-core-spark
```

4.2 Implémentation du moteur DQ — Soda Core

4.2.1 Structure des fichiers du projet DQ

Le projet DQ est organisé selon l'arborescence suivante :

```
1 dq_tiers/
2 |-- dq_tiers_checks.py          # Script principal Python
3 |-- checks/
4 |   |-- checks_kyc.yml          # Regles DQ colonnes KYC
5 |   |-- checks_csp.yml          # Regles DQ colonnes CSP
6 |   +-- checks_pipeline.yml     # Regles fraicheur & pipeline
7 +-- utils/
8     +-- hive_writer.py          # Module d'écriture resultats dans
        Hive
```

4.2.2 Fichier de configuration YAML — Règles KYC (checks_kyc.yml)

```
1 # checks_kyc.yml
2 # Regles Data Quality - Domaine KYC
3 # Source : raw_nova_referentiel_tiers.ccm_npdescription (zone RAW)
4 # Zone RAW = copie exacte d'Oracle NOVA, aucune transformation
   appliquée
5
6 checks for ccm_npdescription
7
8 # --- Completude ---
9 - missing_count(name) = 0
10     name "KYC-01 | Completude | Nom"
11     fail error
12     attributes
13         criticite CRITIQUE
14         domaine KYC
15         dimension completeness
16
17 - missing_count(first_name) = 0
18     name "KYC-02 | Completude | Prenom"
19     fail error
20     attributes
21         criticite CRITIQUE
22         domaine KYC
```

```

23     dimension completeness
24
25 - missing_count(birthdate) = 0
26   name "KYC-03 | Completude | Date de naissance"
27   fail error
28   attributes
29     criticite CRITIQUE
30     domaine KYC
31     dimension completeness
32
33 - missing_count(nationality_country) = 0
34   name "KYC-04 | Completude | Nationalite"
35   fail warn
36   attributes
37     criticite HAUTE
38     domaine KYC
39     dimension completeness
40
41 # --- Validite ---
42 - invalid_count(civility) = 0
43   valid values ["M.", "MME", "MLLE"]
44   name "KYC-05 | Validite | Civilite hors referentiel"
45   fail warn
46   attributes
47     criticite HAUTE
48     domaine KYC
49     dimension validity
50
51 - invalid_count(gender) = 0
52   valid values ["M", "F"]
53   name "KYC-06 | Validite | Genre hors referentiel"
54   fail warn
55   attributes
56     criticite HAUTE
57     domaine KYC
58     dimension validity
59
60 # --- Fraicheur ---
61 - freshness(batch_datetime_eaac4) < 26h
62   name "KYC-07 | Fraicheur | Retard batch"
63   fail error
64   attributes
65     criticite HAUTE
66     domaine PIPELINE
67     dimension freshness

```

4.2.3 Fichier de configuration YAML — Règles CSP (checks_csp.yml)

```

1 # checks_csp.yml
2 # Regles Data Quality - Domaine CSP
3 # Source : raw_nova_referentieltiers.ccm_job (zone RAW)

```

```

4 # Zone RAW = copie exacte d'Oracle NOVA, aucune transformation
   appliquee
5
6 checks for ccm_job
7
8 # --- Completude triple CSP ---
9 - missing_percent(professional_sector) < 5
10     name "CSP-01 | Completude | Secteur professionnel"
11     fail warn
12     attributes
13         criticite HAUTE
14         domaine CSP
15         dimension completeness
16
17 - missing_percent(occupation) < 5
18     name "CSP-02 | Completude | Code profession"
19     fail warn
20     attributes
21         criticite HAUTE
22         domaine CSP
23         dimension completeness
24
25 - missing_percent(socioprofessional_category) < 5
26     name "CSP-03 | Completude | Code CSP"
27     fail warn
28     attributes
29         criticite HAUTE
30         domaine CSP
31         dimension completeness
32
33 # --- Validdite ---
34 - invalid_count(professional_situation) = 0
35     valid values ["SAL", "FON", "TRI", "AUT", "EMP", "NDE"]
36     name "CSP-04 | Validdite | Situation professionnelle"
37     fail warn
38     attributes
39         criticite HAUTE
40         domaine CSP
41         dimension validity
42
43 # --- Conformite PPE ---
44 - missing_count(politically_exposed) = 0
45     name "CSP-05 | Completude | PPE (Personne Politiquement
46     Exposee)"
47     fail error
48     attributes
49         criticite CRITIQUE
50         domaine CSP
51         dimension completeness

```

4.2.4 Script Python principal (dq_tiers_checks.py)

```

1 # dq_tiers_checks.py
2 # Pipeline de Data Quality - Referentiel Tiers PP Clients - Zone
  RAW
3 # CIH Bank - Data Office
4 # Auteur : Saad BENDAHOU
5 # Perimetre : base raw_nova_referentieltiers (HDFS /data/raw/
  novatiers/)
6 # Declenchement : post-ingestion DAG dailyow22 (22h00)
7
8 import argparse
9 from datetime import date
10 from pyspark.sql import SparkSession
11 from soda.scan import Scan
12
13 def run_dq_checks(run_date: str):
14     """
15     Execute les controles DQ Soda Core sur les tables Hive
16     du domaine Tiers et persiste les resultats.
17     """
18     # --- 1. Initialisation de la SparkSession ---
19     spark = SparkSession.builder \
20         .appName("DQ_Tiers_Checks") \
21         .enableHiveSupport() \
22         .getOrCreate()
23
24     # --- 2. Initialisation du scan Soda Core ---
25     scan = Scan()
26     scan.set_data_source_name("hive_raw")
27
28     # Connexion Soda --> Hive RAW via Spark
29     # Base cible : raw_nova_referentieltiers (copie exacte NOVA)
30     from soda.core.spark_df_data_source import SparkDfDataSource
31     spark_source = SparkDfDataSource(
32         data_source_name="hive_raw",
33         spark_session=spark
34     )
35     scan.add_spark_session(spark, "hive_raw")
36
37     # --- 3. Chargement des fichiers de regles YAML ---
38     scan.add_sodacl_yaml_file("checks/checks_kyc.yml")
39     scan.add_sodacl_yaml_file("checks/checks_csp.yml")
40     scan.add_sodacl_yaml_file("checks/checks_pipeline.yml")
41
42     # --- 4. Execution des checks ---
43     scan.execute()
44
45     # --- 5. Collecte et persistance des resultats ---
46     results = scan.get_scan_results()
47     from utils.hive_writer import write_scores, write_anomalies
48     write_scores(spark, results, run_date, dag_name="dailyow22")
49     write_anomalies(spark, results, run_date)
50
51     spark.stop()

```

```

52     print(f"[DQ Tiers] Controles termines pour la date {run_date}")
53
54 if __name__ == "__main__":
55     parser = argparse.ArgumentParser()
56     parser.add_argument("--run_date", type=str,
57                         default=str(date.today()))
58     args = parser.parse_args()
59     run_dq_checks(args.run_date)

```

4.3 Création des tables Hive de persistance

Les tables de persistance DQ sont créées dans la base Hive dq_results. Le script HiveQL de création est le suivant :

```

1  -- Creation de la base dediee DQ
2  CREATE DATABASE IF NOT EXISTS dq_results
3  COMMENT 'Base de resultats Data Quality - CIH Bank Data Office';
4
5  -- Table des scores DQ historises
6  CREATE TABLE IF NOT EXISTS dq_results.dq_tiers_scores (
7      dag_name          STRING,
8      table_name        STRING,
9      column_name       STRING,
10     check_name         STRING,
11     dimension          STRING,
12     criticite          STRING,
13     domaine            STRING,
14     metric_value       DOUBLE,
15     threshold          DOUBLE,
16     passed             BOOLEAN,
17     score_dq           DOUBLE
18 )
19 PARTITIONED BY (run_date DATE)
20 STORED AS ORC
21 TBLPROPERTIES ('orc.compress'='SNAPPY');
22
23 -- Table du detail des anomalies
24 CREATE TABLE IF NOT EXISTS dq_results.dq_tiers_anomalies (
25     table_name          STRING,
26     column_name         STRING,
27     check_name          STRING,
28     identifieur         STRING,
29     radical             STRING,
30     anomaly_detail      STRING,
31     criticite           STRING
32 )
33 PARTITIONED BY (run_date DATE)
34 STORED AS ORC
35 TBLPROPERTIES ('orc.compress'='SNAPPY');

```

4.4 Intégration dans le DAG Airflow dailyow22

4.4.1 Ajout de la tâche DQ

La tâche de Data Quality est ajoutée au DAG `dailyow22` en tant qu'opérateur Python, déclenchée après la fin de toutes les tâches d'ingestion Sqoop des tables `CCM_*`. Le `TriggerRule.ALL_DONE` garantit que les contrôles DQ s'exécutent même si certaines tâches d'ingestion ont échoué partiellement.

```
1 # Extrait du DAG dailyow22 (Airflow)
2 from airflow.operators.python import PythonOperator
3 from airflow.utils.trigger_rule import TriggerRule
4 import subprocess
5
6 def run_dq_tiers(**context):
7     run_date = context['ds'] # Format YYYY-MM-DD
8     result = subprocess.run(
9         [
10             'python',
11             '/opt/airflow/scripts/dq/dq_tiers_checks.py',
12             '--run_date', run_date
13         ],
14         capture_output=True,
15         text=True
16     )
17     if result.returncode != 0:
18         raise Exception(f"DQ Tiers failed:\n{result.stderr}")
19
20 dq_tiers_task = PythonOperator(
21     task_id='run_dq_tiers_checks',
22     python_callable=run_dq_tiers,
23     trigger_rule=TriggerRule.ALL_DONE,
24     dag=dag
25 )
26
27 # Positionnement dans le graphe : apres toutes les ingestions CCM_*
28 [ccm_party_task, ccm_npdescription_task, ccm_job_task,
29  ccm_portfolio_task, ccm_context_task] >> dq_tiers_task
```

4.4.2 Planification et monitoring

Le DAG `dailyow22` s'exécute quotidiennement à **22h00**. La tâche DQ `run_dq_tiers_checks` se déclenche donc automatiquement chaque nuit après l'ingestion, sans intervention manuelle. Les logs Airflow permettent de suivre l'exécution et de détecter toute anomalie dans le pipeline DQ lui-même.

4.5 Création des vues Dremio

Trois vues SQL sont créées dans Dremio, dans le nouveau schéma `dq-tiers`, pointant vers les tables Hive de la base `dq_results`.

4.5.1 Vue v.dq_scores_history

```
1  -- Vue Dremio : historique des scores DQ
2  CREATE OR REPLACE VIEW "dq-tiers"."v.dq_scores_history" AS
3  SELECT
4      run_date,
5      dag_name,
6      table_name,
7      column_name,
8      check_name,
9      dimension,
10     criticite,
11     domaine,
12     metric_value,
13     threshold,
14     passed,
15     score_dq
16 FROM hive.dq_results.dq_tiers_scores
17 ORDER BY run_date DESC, criticite ASC;
```

4.5.2 Vue v.dq_kpis_summary

```
1  -- Vue Dremio : KPIs agregés par domaine et dimension
2  CREATE OR REPLACE VIEW "dq-tiers"."v.dq_kpis_summary" AS
3  SELECT
4      run_date,
5      domaine,
6      dimension,
7      criticite,
8      COUNT(*)                                AS nb_checks,
9      SUM(CASE WHEN passed THEN 1
10             ELSE 0 END)                       AS nb_checks_ok,
11      AVG(score_dq)                             AS score_moyen,
12      MIN(score_dq)                             AS score_min
13 FROM hive.dq_results.dq_tiers_scores
14 GROUP BY run_date, domaine, dimension, criticite
15 ORDER BY run_date DESC, domaine, criticite;
```

4.5.3 Vue v.dq_anomalies_detail

```
1  -- Vue Dremio : detail des anomalies pour Data Stewards
2  CREATE OR REPLACE VIEW "dq-tiers"."v.dq_anomalies_detail" AS
3  SELECT
4      run_date,
5      table_name,
6      column_name,
7      check_name,
8      identifier,
9      radical,
```

```

10     anomaly_detail,
11     criticite
12 FROM hive.dq_results.dq_tiers_anomalies
13 ORDER BY run_date DESC,
14         CASE criticite
15             WHEN 'CRITIQUE' THEN 1
16             WHEN 'HAUTE'    THEN 2
17             WHEN 'MOYENNE'  THEN 3
18             ELSE 4
19         END;

```

4.6 Réalisation du dashboard Tableau

4.6.1 Connexion Tableau à Dremio

Le dashboard Tableau est connecté à Dremio via le connecteur ODBC/JDBC Dremio. Les trois vues du schéma `dq-tiers` sont importées comme sources de données dans Tableau : `v.dq_scores_history`, `v.dq_kpis_summary` et `v.dq_anomalies_detail`.

4.6.2 Onglet 1 — Vue Exécutive

La figure 4.1 présente l'onglet « Vue Exécutive » du dashboard.

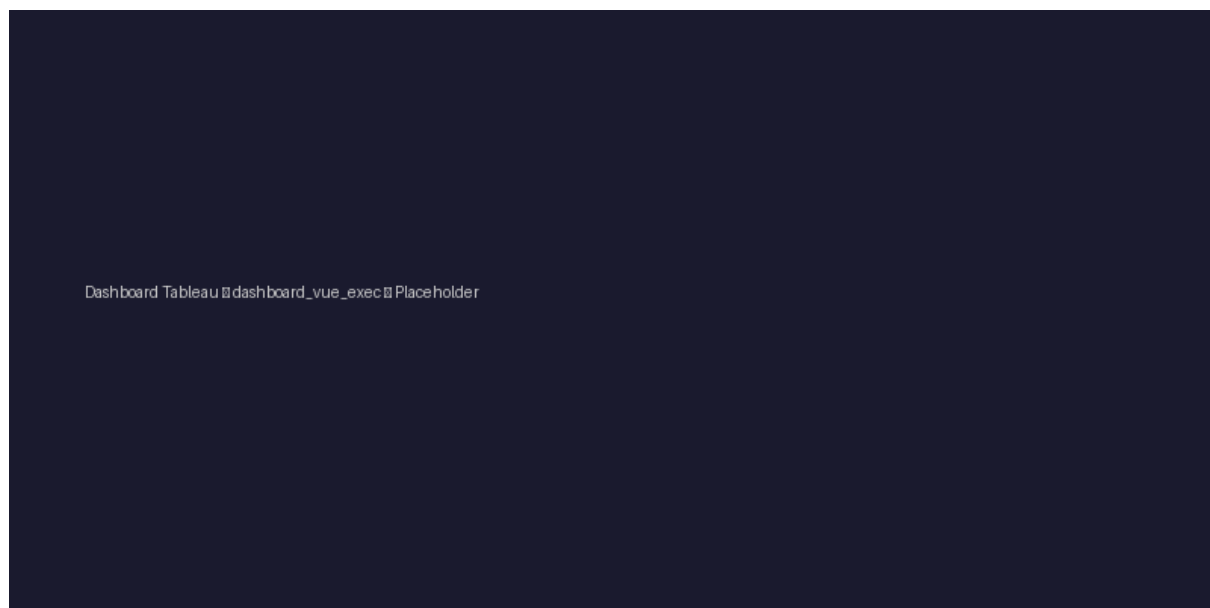


FIGURE 4.1 – Dashboard Tableau — Vue Exécutive : scores DQ globaux KYC et CSP

Cet onglet affiche :

- Le score DQ global du jour pour le domaine KYC et le domaine CSP (indicateurs chiffrés 0–100)
- La courbe d'évolution des scores sur les 30 derniers jours
- Le nombre d'alertes critiques et hautes en cours
- La date et l'heure de la dernière exécution du batch (`dailyow22`)

4.6.3 Onglet 2 — Détail KYC

La figure 4.2 présente l'onglet « Détail KYC ».

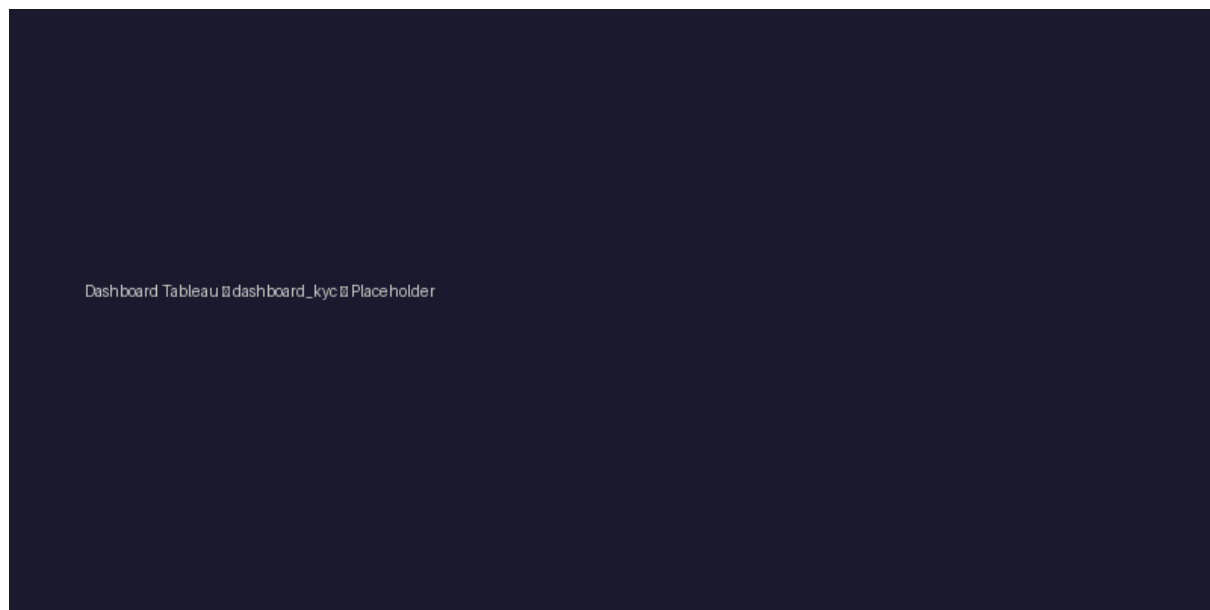


FIGURE 4.2 – Dashboard Tableau — Détail KYC : score par colonne et dimension DQ

4.6.4 Onglet 3 — Détail CSP

La figure 4.3 présente l'onglet « Détail CSP ».

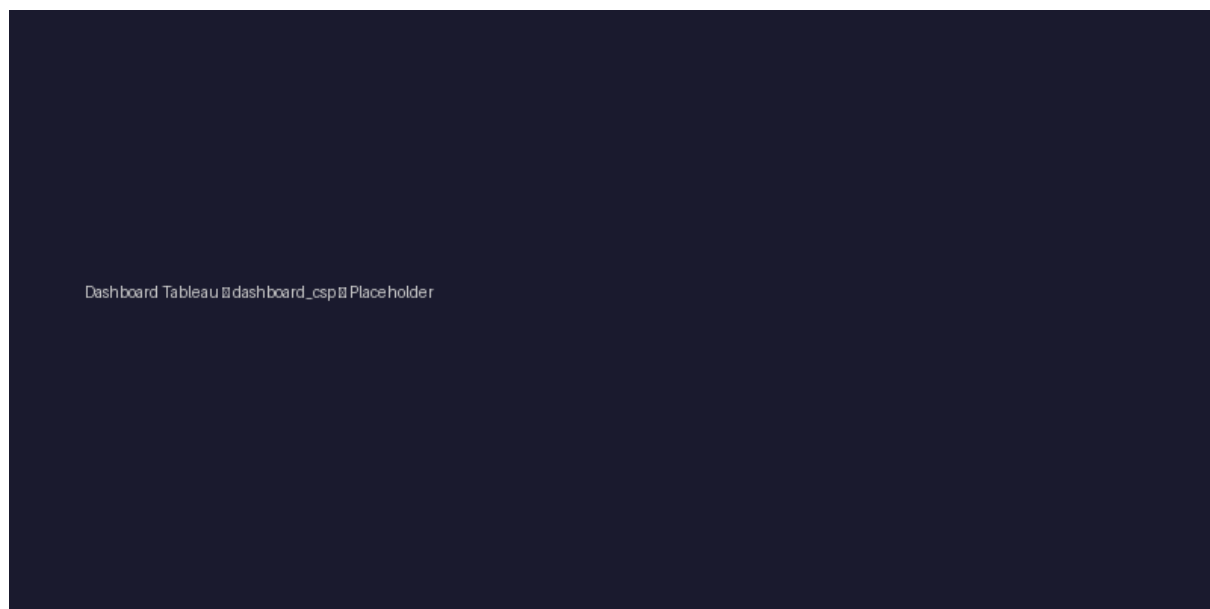


FIGURE 4.3 – Dashboard Tableau — Détail CSP : taux de nullité et valeurs parasites

4.6.5 Onglet 4 — Anomalies et Remédiation

Le quatrième onglet présente la liste paginée des enregistrements clients en anomalie, triée par criticité décroissante. Elle est directement exportable en Excel par les Data Stewards pour engagement d'actions correctives dans NOVA. Les colonnes affichées sont :

`radical` (numéro client CIH), `table_name`, `column_name`, `anomaly_detail`, `criticite`, `run_date`.

Conclusion

Ce chapitre a présenté l'implémentation concrète des différents composants du pipeline de Data Quality Tiers : le script Soda Core avec ses fichiers de configuration YAML, les tables Hive de persistance, la tâche Airflow post-ingestion, les vues Dremio d'exposition et le dashboard Tableau de visualisation. L'ensemble de ces composants forme un pipeline automatisé, opérationnel chaque nuit à 22h, qui permet de mesurer, historiser et restituer la qualité des données KYC et CSP du Référentiel Tiers. Le chapitre suivant présente les résultats obtenus lors des premières exécutions. Chapitre 5 : Résultats et analyse

Conclusion générale